

# Informe N° 3.

Alex Nuñez & Camilo Pinto & Lucas Gonzalez

Octubre de 2025

## Índice

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>1. Descripción del sistema</b>                      | <b>2</b> |
| <b>2. Objetivos esperados</b>                          | <b>2</b> |
| <b>3. Perfiles de usuario</b>                          | <b>2</b> |
| <b>4. Requerimientos</b>                               | <b>3</b> |
| 4.1. Requerimientos no funcionales . . . . .           | 3        |
| 4.2. Trazabilidad u objetivos . . . . .                | 4        |
| <b>5. Modelo de datos y Persistencia</b>               | <b>4</b> |
| 5.1. Interfaz del Servicio de Notificaciones . . . . . | 7        |
| <b>6. Procesos cliente (CLI)</b>                       | <b>7</b> |
| <b>7. Servicios implementados</b>                      | <b>8</b> |
| <b>8. Comentarios</b>                                  | <b>9</b> |

## 1. Descripción del sistema

Se desarrollará un sistema que centraliza la gestión de solicitudes internas mediante tickets, permitiendo registrar incidentes/requerimientos con título, descripción, categoría, prioridad y adjuntos, para darles seguimiento trazable hasta su cierre.

El sistema incluirá autenticación y roles, creación y asignación de tickets, ciclo de vida (Nuevo → En progreso → Resuelto → Cerrado), comentarios e historial de cambios; además, listados con filtros (estado, prioridad, categoría, técnico, fechas) y un tablero para visualizar la carga. Generará un reporte resumido (resueltos/cerrados por categoría), enviará notificaciones en eventos, permitirá la reapertura si el problema persiste, además de la eliminación/anulación por el solicitante cuando corresponda.

El cliente principal, para esta entrega, es por línea de comandos (CLI). No hay interfaz web. Los servicios (Auth, Tickets y Notificaciones) se comunican mediante un bus de servicios externo desplegado con Docker (jrgiadach/soabus) por TCP puerto 5000.

## 2. Objetivos esperados

1. Gestionar tickets de ayuda (help desk): Permitir la creación, seguimiento, asignación, y cierre de tickets.
2. Mejorar la comunicación y resolución de problemas: Facilitar la interacción entre usuarios y administradores, y la posible reapertura de tickets.
3. Proporcionar un control y visibilidad del estado de los problemas: Ofrecer funcionalidades de listado, filtros, y reportes resumidos para el seguimiento de los tickets.
4. Optimizar la atención al usuario: A través de notificaciones y la capacidad de eliminación de tickets por parte del solicitante.

## 3. Perfiles de usuario

El sistema está dirigido a los colaboradores internos de la organización y define tres roles de usuario principales, cada uno con funcionalidades específicas:

1. Usuario: Es el rol estándar para todos los colaboradores. Pueden crear nuevos tickets para reportar incidentes o hacer requerimientos, añadir comentarios, adjuntar archivos, dar seguimiento al estado de sus solicitudes, y reabrir o anular sus propios tickets.
2. Coordinador/Técnico: Es el usuario encargado de resolver los tickets. Tiene la capacidad de visualizar el tablero con la carga de trabajo, tomar la asignación de tickets, cambiar su estado (e.g., a "En Progreso" o "Resuelto"), y comunicarse con los solicitantes a través de comentarios.

3. Administrador: Posee el nivel de acceso más alto. Además de las funcionalidades de los otros roles, puede gestionar usuarios, categorías, y tiene acceso a todos los tickets y reportes para obtener una visión global del sistema, lo que le permite tomar decisiones estratégicas.

## 4. Requerimientos

1. Autenticación y roles: login/logout, recuperación de contraseña, control de acceso por rol (Administrador/Coordinador/Usuario), bloqueo por intentos y expiración de sesión.
2. Gestión de tickets (CRUD + ciclo de vida): crear/ver/actualizar/eliminación lógica con auditoría, adjuntos y comentarios; asignación a coordinadores; estados Nuevo → En progreso → Resuelto → Cerrado; reapertura desde Resuelto/Cerrado; registro de autor, fecha y cambios.
3. Listado y filtros: listar y filtrar por estado, prioridad, categoría, asignado y fechas; paginación, ordenamiento y exportación CSV.
4. Reporte resumen: gráfico de tickets Resueltos/Cerrados por categoría y período; indicadores de tiempo de resolución y tasa de reapertura; acceso según rol.
5. Notificaciones: in-app y correo en creación, asignación, cambio de estado, comentarios y cierre; cola con reintentos y envío diferido si no hay conexión.
6. Acceso y comunicación: cliente por línea de comandos (CLI); autenticación en terminal; interacción entre servicios vía bus externo.
7. Arquitectura y protocolos: SOA con bus externo (jrgiadach/soabus) por TCP 5000; servicios desacoplados que publican/consumen mensajes; cliente CLI; variables `BUS_HOST=soabus` y `BUS_PORT=5000`; contenedores Docker.
8. Geolocalización y riesgos: geolocalización opcional con consentimiento (uso de permisos de navegador o ingreso manual con pin en mapa).
9. Asignación automática de tickets: búsqueda de asignaciones de los operarios en base de datos, prioridad de asignación según actividad de operario y parámetros binarios; se implementará mediante el uso de FIFO y equilibradores de carga (borrador: “asigna mediante reglas de negocio” — categoría, prioridad, ubicación — y notifica al administrador y al usuario de asignación).

### 4.1. Requerimientos no funcionales

1. Disponibilidad: servicio operativo con mantenciones planificadas.
2. Seguridad: todo por conexiones seguras con cifrado vigente; hash seguro; autorización por rol; protección CSRF; límite de consultas; auditoría.

3. Observabilidad: logs estructurados, métricas y trazas distribuidas.
4. Cumplimiento y privacidad: ley chilena de datos personales; consentimiento para ubicación.
5. Límites y restricciones: usuarios, tickets, tamaño de adjuntos y almacenamiento documentados.
6. Riesgos y mitigación: copias de seguridad, reintentos de conexión.

## 4.2. Trazabilidad u objetivos

1. RF Autenticación y roles → Objetivo de control de acceso.
2. RF Gestión de tickets → Objetivo de gestión y trazabilidad.
3. RF Asignación → Objetivo de resolución eficiente.
4. RF Comentarios y adjuntos → Objetivo de colaboración.
5. RF Listado y filtros → Objetivo de visibilidad del estado.
6. RF Reporte resumen → Objetivo de control y decisiones.
7. RF Notificaciones → Objetivo de atención oportuna.
8. RF Acceso y comunicación → Objetivo de disponibilidad a usuarios.
9. RF Geolocalización → Objetivo de análisis y despacho.

## 5. Modelo de datos y Persistencia

Mecanismo de Persistencia: Para el almacenamiento de los datos se utilizará un sistema de gestión de bases de datos relacional (RDBMS). Esta elección se debe a la naturaleza estructurada de los datos, la necesidad de mantener la integridad referencial entre entidades como tickets, usuarios y comentarios, y las capacidades transaccionales que ofrece.

1. Tickets:
  - ID (int pk, autoincrement).
  - Título (varchar(120) not null).
  - Descripción (text not null).
  - categoria\_id (int fk → categorias.id, ON DELETE RESTRICT).
  - Prioridad (enum: BAJA, MEDIA, ALTA, CRITICA).
  - Estado (enum: NUEVO, EN\_PROGRESO, RESUELTO, CERRADO).
  - Fecha creación (datetime default CURRENT\_TIMESTAMP).
  - Fecha cierre (datetime null).

- solicitante\_id (int fk → usuarios.id, ON DELETE RESTRICT).
- ejecutivo\_id (int fk → usuarios.id, ON DELETE SET NULL).
- Índices: (estado, prioridad), categoria\_id, solicitante\_id, ejecutivo\_id.

## 2. Usuarios:

- ID (int pk, autoincrement).
- Nombre (varchar(120) not null).
- Email (varchar(120) unique not null).
- password\_hash (char(60) not null).
- Rol (enum: ADMIN, COORDINADOR, USUARIO).
- Fecha creación (datetime default CURRENT\_TIMESTAMP).
- Activo (tinyint(1) default 1).

## 3. Categorías:

- ID (int pk, autoincrement).
- Nombre (varchar(80) unique not null).
- Descripción (text null).

## 4. Comentarios:

- ID (int pk, autoincrement).
- Contenido (text not null).
- Fecha creación (datetime default CURRENT\_TIMESTAMP).
- ticket\_id (int fk → tickets.id, ON DELETE CASCADE).
- usuario\_id (int fk → usuarios.id, ON DELETE SET NULL).
- Índice: ticket\_id.

## 5. Adjuntos:

- ID (int pk, autoincrement).
- Nombre (varchar(120) not null).
- Ruta (varchar(255) not null).
- Tipo (varchar(100) not null).
- Tamaño (int unsigned null).
- Checksum (varchar(64) null).
- Fecha subida (datetime default CURRENT\_TIMESTAMP).
- ticket\_id (int fk → tickets.id, ON DELETE CASCADE).
- usuario\_id (int fk → usuarios.id, ON DELETE SET NULL).
- Índice: ticket\_id.

Conforme a los requisitos del proyecto, el sistema se diseñará bajo una Arquitectura Orientada a Servicios (SOA). Esta arquitectura desacopla las funcionalidades en servicios independientes y reutilizables, que se comunican a través de una red. Los componentes principales de la arquitectura son:

1. Componente Cliente: Aplicación por línea de comandos que invoca casos de uso y publica/consume mensajes vía bus. Sin UI web en esta entrega.
2. Componentes de Servicio: Son los encargados de implementar la lógica de negocio. Se han definido tres servicios principales:
  - Servicio de Autenticación (Auth): Responsable de gestionar la identidad de los usuarios: login, logout, gestión de sesiones, recuperación de contraseñas y validación de roles y permisos.
  - Servicio de Tickets: Es el servicio principal que encapsula toda la lógica relacionada con la gestión de tickets: creación, actualización, asignación, comentarios, adjuntos, listados y reportes.
  - Servicio de Notificaciones: Opera de forma asíncrona. Su función es gestionar el envío de correos electrónicos y notificaciones in-app, utilizando el bus externo para asegurar la entrega con reintentos incluso si los sistemas externos no están disponibles temporalmente.

Se describen los contratos de mensajería por bus para cada servicio, indicando el requerimiento funcional (RF) que satisfacen.

## Contratos por bus de servicios

En esta entrega los servicios no exponen HTTP. Se comunican por un **bus TCP** externo (jrgiadach/soabus, puerto 5000). El protocolo es *JSON por línea* (JSONL): cada mensaje es un JSON terminado en salto de línea. Hay dos patrones: **request/response** (sincronía punto a punto) y **eventos** (publicación asíncrona).

### Convenciones

- Variables: BUS\_HOST=soabus, BUS\_PORT=5000.
- Campos: type {request|event}, service, action o topic, payload.
- Fiabilidad: el emisor reintenta ante error y el consumidor idempotiza por id de mensaje.

Auth.authenticate (request) Autentica credenciales y retorna perfil.

```
{"type":"request","service":"auth","action":"authenticate",  
  "payload":{"email":"usuario@acme.com","password":"****"}}
```

Tickets.get\_by\_user (request) Lista tickets del usuario.

```
{"type":"request","service":"tickets","action":"get_by_user",  
  "payload":{"user_id":123}}
```

`Ticket.created (event)` Se emite al crear un ticket; lo consume Notificaciones.

```
{"type":"event","topic":"ticket.created",  
  "payload":{"id":101,"categoria":"Soporte","usuario":123}}
```

## 5.1. Interfaz del Servicio de Notificaciones

Este servicio no expone una API pública para el cliente web, sino que se comunica a través del bus externo (jrgiadach/soabus) y aplica reintentos ante fallas externas. Otros servicios (como el de Tickets) publican eventos en el bus externo (e.g., `ticket.created`, `ticket.comment.added`) y el servicio de notificaciones los consume para realizar los envíos.

### 1. Evento: `ticket.created`

Descripción: Se activa al crear un ticket. El servicio envía una notificación por correo al solicitante.

RF Satisfecho: Notificaciones.

RNF Considerado: Disponibilidad (la cola con reintentos mitiga caídas del servidor de correo).

### 2. Evento: `ticket.assigned`

Descripción: Se activa al asignar un ticket a un coordinador. El servicio notifica al usuario asignado.

RF Satisfecho: Notificaciones.

## 6. Procesos cliente (CLI)

### Crear ticket

```
docker compose up -d soabus
```

```
docker compose run --rm -it app python -m Sistema_Ticket_Soa.main
```

```
# Login -> Crear ticket -> completar -> confirmar
```

### Asignar y comentar

```
# Menú -> Asignar ticket -> seleccionar técnico
```

```
# Menú -> Agregar comentario -> escribir texto
```

### Cerrar y reabrir

```
# Menú -> Cambiar estado a Resuelto/Cerrado
```

```
# Menú -> Reabrir si corresponde
```

## Crear ticket

1. Usuario inicia sesión.
2. Abre “Nuevo ticket”, completa y envía.
3. Sistema confirma con ID y estado *Abierto*.

## Asignar y comentar

Pasos para asignar responsable y agregar comentarios con captura de pantalla.

## Cerrar y reabrir

Criterios de cierre, permisos para reabrir y evidencia.

# 7. Servicios implementados

Estado por módulo, contratos y decisiones.

## Resumen

| Módulo         | Descripción                            | Estado               |
|----------------|----------------------------------------|----------------------|
| Autenticación  | Login JWT, roles                       | Implementado/Parcial |
| Tickets        | CRUD, asignación, comentarios, estados | Implementado/Parcial |
| Notificaciones | Email/In-app                           | Implementado/Parcial |

## Contratos por bus de servicios

Los servicios intercambian mensajes JSON por línea (JSONL) en el bus TCP (jrgiadach/soabus).

`Auth.authenticate (request)`

```
{"type": "request", "service": "auth", "action": "authenticate",  
  "payload": {"email": "usuario@acme.com", "password": "****"}}
```

`Tickets.get_by_user (request)`

```
{"type": "request", "service": "tickets", "action": "get_by_user",  
  "payload": {"user_id": 123}}
```

`Ticket.created (event)`

```
{"type": "event", "topic": "ticket.created",  
  "payload": {"id": 101, "categoria": "Soporte", "usuario": 123}}
```



## 8. Comentarios

Se decidió implementar para esta entrega un cliente por línea de comandos en vez de una interfaz web. La razón: bajar complejidad del frontend, menor tiempo requerido para su implementación y validar primero los servicios núcleo (Auth, Tickets, Notificaciones) y el flujo de negocio. El CLI permite crear, listar, asignar, comentar, cerrar y reabrir tickets usando el bus externo por TCP 5000 (jrgiadach/soabus), manteniendo trazabilidad y roles. La UI web queda como trabajo posterior, reutilizando los mismos contratos y agregando pruebas de usabilidad. Este recorte no afecta los RF clave ni los RNF de seguridad, observabilidad y disponibilidad planificados.